PeeGeeq Connection Management and HAProxy Failover

Author: Mark A Ray-Smith Cityline Ltd.
Date: Aug 26, 2025
Version: 1.1 (updated May 6, 2026 — incorporates architecture review)

Executive Summary

PeeGeeQ connects to PostgreSQL exclusively via the Vert.x 5.x reactive client (`io.vertx.pgclient`). No JDBC, no HikariCP, no connection URL strings. Connection options — host, port, database, credentials, and `search_path` — are set individually on `PgConnectOptions`; the pool is built once per `serviceId` and cached in a `ConcurrentHashMap`.

The pool itself has **no built-in failover**. When a backend goes away, Vert.x discards broken connections on the next acquisition attempt and opens new ones. Whether those new connections succeed depends entirely on what the pool's configured host/port resolves to at that moment — which is precisely the problem an external TCP proxy solves.

Design principle confirmed by architecture review: PeeGeeQ deliberately delegates database-tier failover to infrastructure. The application tier handles its own node-level failover (via `ConnectionRouter` + Consul) independently of the database tier. These two planes do not need to know about each other.

This document records:

1. The full connection management architecture as found in the codebase.
2. The full resilience stack active during a failover window.
3. Application-tier vs database-tier failover — how the two planes relate.
4. Why the Vert.x pool requires an external proxy for database failover.
5. The design decisions behind the HAProxy failover integration test.
6. The local developer docker-compose topology (HAProxy + PgBouncer).
7. Known gaps and future work.
8. How to build and use the pg-sidecar service.

Document Map

Section	What it covers	Audience
§1 Connection Management Architecture	How `PgConnectionManager`, `PgClientFactory`, and `PgBuilder.pool()` fit together; configuration loading; pool creation; reconnection; schema isolation	Developers, reviewers
§2 Resilience Stack	The three independent mechanisms active during a failover window: Vert.x pool discard/reconnect, `HealthCheckManager` + circuit breaker, `ConnectionRouter` + Consul	Developers, architects

Section	What it covers	Audience
§3 Application vs Database Failover	Why the two failover planes are orthogonal and what each one handles	Architects, ops
§4 Why an External Proxy Is Needed	Why the Vert.x pool cannot self-heal without HAProxy; how HAProxy fills the gap; Patroni as an optional enhancement	Architects, ops
§5 HAProxy Integration Test	Container topology, test phases, retry design, how to run the test	Developers
§6 Local Developer Environment	Docker-compose stack (HAProxy + PgBouncer); how to simulate a failover with `docker compose stop`	Developers
§7 Known Gaps	Summary of five open gaps; links to the detailed implementation plan	All
§8 Using the pg-sidecar	How to build, configure, and run `peegeeq-pg-sidecar`; HAProxy `httpchk` config; verifying the endpoint	Developers, ops

Resiliency Options at a Glance

The table below shows every PostgreSQL database-tier resiliency option available with PeeGeeQ, ordered from simplest to most complete. Choose the row that matches your reliability requirements, then follow the detail links.

Option	Failover behaviour	Promotion mechanism	Write downtime	Extra infrastructure
Single managed FQDN (DBA/DevOps-controlled)	No automatic failover — DBA/DevOps updates the DNS record or virtual IP to point at the new primary; pool reconnects automatically once the FQDN resolves to the live node	Manual DBA / DevOps (DNS or VIP flip)	Until FQDN is re-pointed	DNS TTL management or virtual IP (no proxy process required)
HAProxy + `pgsql-check`	Stops routing to dead node; standby stays read-only; no auto-promotion	Manual DBA	Until DBA promotes	HAProxy
HAProxy + `httpchk` + `peegeeq-pg-sidecar`	Semantic primary detection via `pg_is_in_recovery()`; no auto-promotion	Manual DBA	Until DBA promotes	HAProxy + sidecar per node
HAProxy + Patroni	Automatic failover; DCS leader election; fencing + `pg_rewind`	Patroni (`pg_promote()` + `pg_ctl`)	~10–30 s	HAProxy + Patroni + DCS (etcd / Consul / ZK)

Option	Failover behaviour	Promotion mechanism	Write downtime	Extra infrastructure
HAProxy + Consul monitor (`peegeeq-service-manager`)	Automatic failover; Consul TTL + LockDelay fencing; partial (`pg_terminate_backend`)	`pg_promote()` via reactive pgclient	~25–30 s (tunable)	Consul cluster (already required by service-manager)

Reading the table

- **Option 1 (single managed FQDN)** — no proxy process at all; the application points at a single FQDN or virtual IP that DBA/DevOps re-points to the new primary after failover. The Vert.x pool reconnects automatically once the FQDN resolves to the live node. Suitable for development and non-critical workloads.
- **Option 2 (HAProxy + `pgsql-check`)** — adds TCP-level health checking and automatic re-routing away from a dead node, but cannot distinguish primary from replica. The standby remains read-only; a DBA must still promote manually.
- **Option 3 (HAProxy + sidecar)** — adds semantic primary detection (`pg_is_in_recovery()`) without requiring Patroni. HAProxy routes only to the true write primary. Correct for production with manual failover.
- **Option 4 (HAProxy + Patroni)** — full Patroni stack; the industry-standard approach for PostgreSQL HA. Requires an external DCS. Provides automatic promotion, fencing, and `pg_rewind` for safe re-join.
- **Option 5 (Consul monitor)** — the native PeeGeeQ path: re-uses the Consul cluster that `peegeeq-service-manager` already requires. Avoids introducing Patroni. The `PgFailoverMonitor` and `PgPrimaryElector` components are **proposed** (see PEEGEEQ_FAILOVER_CONSUL_DESIGN.md).

In all cases, point `peegeeq.database.host` and `peegeeq.database.port` at the proxy or load-balancer endpoint — never directly at a PostgreSQL node.

> **And JDBC?** The PostgreSQL JDBC driver supports a multi-host URL (`jdbc:postgresql://host1,host2/db?targetServerType=primary`) that provides client-side failover without a proxy. This pattern is not applicable to PeeGeeQ — JDBC is a blocking synchronous protocol incompatible with the Vert.x reactive model — and has significant structural weaknesses in distributed systems. See Appendix B for details.

`HealthCheckManager` updates a health status map and optionally short-circuits its own periodic checks via the circuit breaker — it does not stop the `Pool` or gate business operations. The pool continues serving requests throughout the failover window; requests that arrive before HAProxy switches fail, and those after succeed. The circuit breaker in this codebase guards health-check queries, not business queries.

---

1. Connection Management Architecture

1.1 Class hierarchy

```
PeeGeeQManager (public facade)
  └── PgClientFactory
```

```
        └── PgConnectionManager            peegeeq-
db/.../connection/PgConnectionManager.java
            └── PgBuilder.pool()  ──► io.vertx.pgclient  (Vert.x 5.x)
```

1.2 Configuration loading

`PeeGeeQConfiguration` is the single entry point for all configuration. It resolves values through a fixed four-layer priority chain (lowest to highest):

```
1. peegeeq-default.properties        (classpath, always loaded)
2. peegeeq-<profile>.properties      (classpath, profile-specific overrides)
3. Environment variables             (PEEGEEQ_* prefix, converted to property
keys)
4. System properties                 (-Dpeegeeq.* JVM args — highest priority)
```

Profile selection — evaluated once at construction time:

```
System.getProperty("peegeeq.profile")   →  env PEEGEEQ_PROFILE   →  fallback
"default"
```

Named profiles (files in `peegeeq-db/src/main/resources/`):

Profile	File	Tuned for
`default`	`peegeeq-default.properties`	local development
`development`	`peegeeq-development.properties`	local dev variant
`production`	`peegeeq-production.properties`	production workload
`reliable`	`peegeeq-reliable.properties`	durability-first
`high-throughput`	`peegeeq-high-throughput.properties`	throughput-first
`high-performance`	`peegeeq-high-performance.properties`	low-latency
`low-latency`	`peegeeq-low-latency.properties`	low-latency
`extreme-performance`	`peegeeq-extreme-performance.properties`	benchmarks
`vertx5-optimized`	`peegeeq-vertx5-optimized.properties`	Vert.x 5 tuning
`bitemporal-optimized`	`peegeeq-bitemporal-optimized.properties`	bitemporal workload
`parallel-test`	`peegeeq-parallel-test.properties`	parallel test isolation

Default property values (from `peegeeq-default.properties`):

```
# Connection
peegeeq.database.host=localhost
```

```
peegeeq.database.port=5432
peegeeq.database.name=peegeeq
peegeeq.database.username=peegeeq
peegeeq.database.password=peegeeq
peegeeq.database.schema=public
peegeeq.database.ssl.enabled=false

# Pool
peegeeq.database.pool.min-size=8
peegeeq.database.pool.max-size=32
peegeeq.database.pool.shared=true
peegeeq.database.pool.connection-timeout-ms=30000
peegeeq.database.pool.idle-timeout-ms=600000
peegeeq.database.pool.max-lifetime-ms=1800000
peegeeq.database.pool.auto-commit=true
```

Constructor variants of `PeeGeeQConfiguration`:

Constructor	Use case
`PeeGeeQConfiguration()`	Production — auto-detects profile
`PeeGeeQConfiguration(profile)`	Explicit profile
`PeeGeeQConfiguration(profile, Properties overrides)`	Tests — isolated override map, never writes to `System.getProperties()`
`PeeGeeQConfiguration(profile, host, port, name, user, password, schema)`	Testcontainers setup — avoids `System.setProperty` race conditions between parallel tests

After loading, `PeeGeeQConfiguration` passes the resolved `Properties` map to `PgConnectionConfig.Builder` (connection details) and `PgPoolConfig.Builder` (pool sizing), both of which read their specific keys and apply safe defaults for any missing values.

1.3 How a pool is created

`PgConnectionManager.createReactivePool()` (called once per `serviceId` via `computeIfAbsent`):

```
PgConnectOptions connectOptions = new PgConnectOptions()
    .setHost(connectionConfig.getHost())
    .setPort(connectionConfig.getPort())
    .setDatabase(connectionConfig.getDatabase())
    .setUser(connectionConfig.getUsername())
    .setPassword(connectionConfig.getPassword())
    .setSslMode(sslEnabled ? SslMode.REQUIRE : SslMode.DISABLE)
    .setProperties(Map.of("search_path", schema));  // per-tenant schema isolation

PoolOptions poolOptions = new PoolOptions()
    .setMaxSize(poolConfig.getMaxSize())
    .setMaxWaitQueueSize(poolConfig.getMaxWaitQueueSize())
```

```
        .setConnectionTimeout(...)
        .setIdleTimeout(...)
        .setShared(poolConfig.isShared());

    Pool pool = PgBuilder.pool()
        .with(poolOptions)
        .connectingTo(connectOptions)
        .using(vertx)
        .build();
```

`search_path` is injected at the `PgConnectOptions` level so every connection from the pool inherits the correct schema automatically. This is the project's single source of truth for schema isolation; it must never be duplicated in SQL strings.

1.4 Pool configuration defaults

Property	Default (properties file)	Integration-test override
`peegeeq.database.pool.max-size`	32	3
`peegeeq.database.pool.shared`	`true`	`false`
`peegeeq.database.pool.connection-timeout-ms`	30 000 ms	30 000 ms
`peegeeq.database.pool.idle-timeout-ms`	600 000 ms (10 min)	5 000 ms

`PgPoolConfig.Builder` code defaults (before properties are applied): `maxSize=16`, `maxWaitQueueSize=128`, `connectionTimeout=30 s`, `idleTimeout=10 min`, `shared=true`.

1.5 Reconnection behaviour (main pool)

The Vert.x reactive pool does **not** proactively reconnect. When a connection is acquired (`pool.getConnection()`, `pool.withConnection()`, `pool.withTransaction()`) and the underlying TCP socket is dead, Vert.x discards the broken connection and opens a new one to the configured host/port. There is no backoff, no retry loop, and no awareness of primary/replica topology.

Exception — LISTEN/NOTIFY connections (`ReactiveNotificationHandler` in `peegeeq-bitemporal`): these hold a single long-lived connection. When it drops, `ReactiveNotificationHandler` reconnects with exponential backoff:

- `MAX_RECONNECT_ATTEMPTS = 5`
- Base delay: 1 000 ms; formula: `1000 * (1 << min(attempt-1, 5))` → caps at 32 s
- Resets attempt counter to 0 on success
- Guarded by `shutdown` flag — no reconnect during intentional close

1.6 Schema isolation

`peegeeq.database.schema` is the single source of truth. `PgConnectionConfig.getSchema()` reads it; `PgConnectionManager.normalizeSearchPath()` validates and normalises the value (letters, digits,

underscore, comma only — no quoted identifiers, no `$user`). The normalised value is then written into `PgConnectOptions.setProperties("search_path", ...)`, making it a per-connection startup parameter rather than a `SET search_path` statement that can be overridden by application SQL.

1.7 Connection string / URL patterns

Usage	Pattern
Vert.x reactive pool	`PgConnectOptions` fields — no URL
Flyway migrations	`jdbc:postgresql://<host>:<port>/<db>?currentSchema=<schema>` via `PgConnectionConfig.getJdbcUrl()`

`getJdbcUrl()` is used **only** by migration tooling. It is never called on the reactive pool path and must not be introduced there.

---

2. Resilience Stack Active During a Failover Window

When the primary PostgreSQL goes down, three independent mechanisms activate in PeeGeeQ. Understanding them separately is important because they operate on different timescales and are not coordinated with each other.

2.1 Vert.x pool — silent discard and reconnect

Timescale: **per-request** (no polling, no background thread).

When `pool.getConnection()` or `pool.withConnection()` is called and the pooled TCP socket is dead, Vert.x discards the broken connection and immediately attempts a new TCP connection to the configured host/port (which, if HAProxy is used, is the proxy address). There is no explicit delay, no backoff, and no counter — it either succeeds or returns a failed `Future`.

This means the Vert.x pool **does not suspend itself** during a failover. It tries every request. Requests that arrive in the first ~1 s after primary failure (before HAProxy has detected and switched) will fail. Requests after the switch will succeed.

2.2 HealthCheckManager + CircuitBreakerManager — detection and status

Timescale: **periodic** (configured interval, default every 30 s in production).

`HealthCheckManager` runs `SELECT 1` against the pool on a `Vertx.setPeriodic()` timer. Each check is optionally wrapped with a `Resilience4j CircuitBreaker` via `CircuitBreakerManager.getCircuitBreaker(name)`:

```
// In HealthCheckManager.executeWithCircuitBreaker():
if (circuitBreakerManager != null) {
    CircuitBreaker cb = circuitBreakerManager.getCircuitBreaker(cbName);
    if (cb.getState() == CircuitBreaker.State.OPEN) {
        return Future.succeededFuture(HealthStatus.unhealthy(cbName, "Circuit
breaker open"));
```

```
    }
}
```

When health checks start failing after a primary outage:

1. `HealthCheckManager` marks the database component `UNHEALTHY` in its `lastResults` map.
2. The `OverallHealthInfo` returned by `getHealth()` reflects `DOWN`.
3. The circuit breaker (if configured) opens after its threshold is reached, causing subsequent checks to short-circuit to `UNHEALTHY` without hitting the pool.

Important nuance: `HealthCheckManager` marks status; it does **not** stop the pool or prevent application code from calling `pool.withConnection()`. Business operations and health checks share the same pool independently.

2.3 Application-tier failover — ConnectionRouter + Consul

Timescale: **per-request on the application routing layer** (independent of database tier).

This mechanism is entirely separate from database failover. It handles failover between **PeeGeeQ application instances** in a multi-node deployment, not between PostgreSQL nodes.

`ConnectionRouter` routes incoming client requests (from microservices or the Management UI) to a healthy PeeGeeQ instance discovered via `ConsulServiceDiscovery`:

```
// In ConnectionRouter.routeGetRequest():
serviceDiscovery.discoverInstances()
    .compose(instances -> {
        PeeGeeQInstance selected = loadBalancer.selectInstance(instances,
environment, region);
        if (selected == null) {
            return Future.failedFuture("No healthy instances available");
        }
        return routeRequestWithRetry(selected, path, instances, environment,
region, 0);
    });
```

`LoadBalancer` supports `ROUND_ROBIN` and other strategies. `routeRequestWithRetry` attempts up to `maxRetries` (default 3) additional instances before failing.

When a PeeGeeQ instance loses its database connection and its `/health` endpoint returns `DOWN`, Consul deregisters it. `ConsulServiceDiscovery.discoverInstances()` then stops returning that instance, and `ConnectionRouter` automatically routes to the remaining healthy instances.

Key distinction: Consul-based application failover makes no assumptions about which PostgreSQL node is primary. Each surviving PeeGeeQ instance manages its own pool independently.

2.4 Combined failover sequence (HAProxy + Consul cluster)

```
t=0 s    Primary PostgreSQL goes down.

t≈0 s    Vert.x pools on all instances: in-flight requests fail immediately.
          Business operations receive Future failures.

t≈1 s    HAProxy detects primary down (fall=2 × inter=500ms).
          HAProxy starts routing new TCP connections to secondary (backup).
          Vert.x pools: new connection attempts now succeed to secondary.
          Business operations resume successfully.

t≈30 s   HealthCheckManager periodic check fires on each instance.
          SELECT 1 succeeds (now reaching secondary via HAProxy).
          Health status returns to UP.
          Circuit breaker (if configured) resets.

Consul:   /health endpoint on each instance reflects UP again.
          No Consul deregistration occurs if instances recovered within the
          Consul TTL window.

t=N      Primary returns. HAProxy detects recovery (rise=1, ~500 ms).
          Traffic automatically fails back to primary.
          Vert.x pool opens new connections to primary.
```

---

3. Application-Tier Failover vs Database-Tier Failover

These are orthogonal concerns. The table below maps the two planes:

Concern	Mechanism	Scope
PostgreSQL primary → secondary	HAProxy / VIP / RDS endpoint	Database tier, infrastructure
Pool reconnection	Vert.x pool implicit per-request	Database client tier
Health status reporting	`HealthCheckManager` + `CircuitBreakerManager`	Application tier, observability
PeeGeeQ instance A → instance B	`ConnectionRouter` + Consul	Application tier, routing

A production deployment must address **both** planes:

1. Place an HAProxy, Patroni VIP, or managed service writer endpoint between PeeGeeQ and PostgreSQL. Set `peegeeq.database.host` to that proxy/VIP address.
2. Run multiple PeeGeeQ instances registered with Consul. `ConnectionRouter` handles application-level failover automatically.

Neither plane knows about the other, and they do not need to.

---

4. Why the Vert.x Pool Needs an External Proxy for Failover

`PgBuilder.pool().connectingTo(connectOptions)` bakes a single host/port into the pool at construction time. Changing where that pool connects requires rebuilding it — which means restarting the `PeeGeeQManager`.

An external TCP proxy (`HAProxy`) solves this at the network layer:

```
Vert.x pool
  │  host=haproxy, port=5400   ← fixed, never changes
  ▼
HAProxy
  ├── pg_primary:5432   (active, health-checked every 500 ms)
  └── pg_secondary:5432 (backup — activated when primary is DOWN)
```

When the primary fails:

1. HAProxy detects failure after 2 consecutive failed checks (~1 s with `fall=2 inter=500ms`).
2. HAProxy promotes the secondary backup and starts routing new TCP connections to it.
3. The Vert.x pool's existing connections to the dead primary fail on next use; Vert.x discards them and opens new connections — which HAProxy now routes to the secondary.
4. **No application restart. No pool rebuild. No code change.**

Auto-recovery: when primary returns, HAProxy detects recovery after 1 successful check (`rise=1`) and resumes routing to it; the backup reverts to standby.

Why the same port on both backends is not a problem: HAProxy distinguishes backends by **hostname or IP address**, not by port number. Port 5432 is the PostgreSQL wire-protocol port on each server; the servers themselves are separate hosts (containers, VMs, or physical machines) that resolve to different IP addresses on the Docker network. HAProxy maintains independent TCP connections to each backend's `host:port` pair and health-checks them separately. The client connecting to `haproxy:5400` only ever sees the single proxy address; the fact that both backends share port 5432 is irrelevant. This is identical to how an HTTP load balancer works with two web servers both listening on port 443 — the LB routes by upstream IP, not by upstream port.

HAProxy does not require Patroni: the two are independent tools. HAProxy alone, using plain TCP health checks (`check inter 500ms`), is sufficient to detect a dead primary and promote the backup — no leader-election daemon needed. This is exactly how the test and local docker-compose stacks in this project work.

Patroni is an optional enhancement. When Patroni manages the PostgreSQL cluster, it exposes a REST API on each node (default port 8008). HAProxy can use this as a more semantically precise health check:

```
# Patroni-aware health check — only routes to the Patroni-elected write primary
server pg1 pg1:5432 check port 8008 httpchk GET /primary
server pg2 pg2:5432 check port 8008 httpchk GET /primary backup
```

With this config, HAProxy only routes to the node Patroni has promoted as primary, rather than any node that accepts a TCP connection. This prevents accidentally routing writes to a replica that is technically up but not the write primary. For the current project setup (test + local dev), plain TCP checks are sufficient.

---

5. HAProxy Failover Integration Test

5.1 Files created

File	Purpose
`peegeeq-db/src/test/resources/haproxy-failover.cfg`	HAProxy config embedded in the JAR, mounted into the container at test startup
`peegeeq-db/src/test/java/.../resilience/HaProxyConnectionFailoverTest.java`	The integration test

5.2 Container topology (Testcontainers)

```
Docker bridge network (created per test class run)
  │
  ├── pg_primary   (PostgreSQLContainer, alias "pg_primary")
  ├── pg_secondary (PostgreSQLContainer, alias "pg_secondary")
  └── haproxy      (GenericContainer "haproxy:2.8-alpine")
        │  mounts haproxy-failover.cfg from classpath
        └── exposes HAPROXY_PG_PORT:5400 → random host port
```

`PgConnectionManager` is pointed at `haproxy.getHost() : haproxy.getMappedPort(5400)`. It never knows the actual PostgreSQL host/port.

5.3 Test phases

Phase 1 — Normal operation (`@Order(1)`)

- Executes `SELECT 1` through the pool.
- Verifies HAProxy routes to the primary and the pool functions normally.

Phase 2 — HAProxy TCP failover (`@Order(2)`)

1. Confirms pre-failover query succeeds.
2. Calls `primary.stop()` to kill the primary container.
3. Waits 4 s (HAProxy detection time ~1 s + 3 s buffer).
4. Retries `SELECT 1` up to 8 times at 1 s intervals.
5. Asserts the query eventually returns 1, routed through HAProxy to the secondary.

> **Production note**: in production, primary and secondary would share data via PostgreSQL streaming replication. This test uses two independent nodes because it targets *connection-level* resilience, not data consistency.

5.4 Retry design (no `.recover()`)

`.recover()` is banned in this codebase (see `docs-design/code reviews/vertx-recover-usage.md`). The retry loop uses `Promise<Integer>` + `vertx.setTimer()` + `.onSuccess()` / `.onFailure()`:

```
private void scheduleAttempt(Vertx vertx, Pool pool, int maxAttempts, int attempt,
                             long delayMs, Promise<Integer> result) {
    pool.query("SELECT 1 AS health").execute()
        .onSuccess(rows ->
result.complete(rows.iterator().next().getInteger("health")))
        .onFailure(err -> {
            if (attempt >= maxAttempts) {
                result.fail(err);
            } else {
                vertx.setTimer(delayMs, id ->
                    scheduleAttempt(vertx, pool, maxAttempts, attempt + 1,
delayMs, result));
            }
        });
}
```

This is correct Vert.x 5.x style: state is carried in a `Promise`, control flow in `.onSuccess()`/`.onFailure()`, delays via `vertx.setTimer()`.

5.5 Running the test

```
mvn test -pl peegeeq-db -Pintegration-tests '-Dtest=HaProxyConnectionFailoverTest'
```

Requirements: Docker Desktop running. The test pulls `haproxy:2.8-alpine` on first run.

5.6 HAProxy config (annotated)

```
global
    maxconn 200
    log     stdout format raw local0 info

defaults
    mode              tcp          # PostgreSQL is a raw TCP protocol
    timeout connect   5s
    timeout client    60s
    timeout server    60s
    option            tcplog
```

```
frontend pg_frontend
    bind *:5400
    default_backend pg_backends

backend pg_backends
    balance           leastconn   # good for long-lived PG connections

    # pgsql-check sends a PostgreSQL startup packet to each backend.
    # The server responds with an authentication challenge; any response
    # (including "wrong password") proves PostgreSQL is alive and serving
    # the wire protocol — not just accepting a TCP connection.
    # tcp-check would pass even if PostgreSQL is still in crash recovery or
    # max_connections is exhausted.
    # Requires a 'haproxy_check' user to exist in PostgreSQL (no password needed).
    option            pgsql-check user haproxy_check

    # pg_primary / pg_secondary are Docker network aliases on the test network.
    # fall=2: mark DOWN after 2 consecutive failed checks (~1 s at inter=500ms)
    # rise=1: recover after 1 successful check
    server pg_primary   pg_primary:5432   check inter 500ms fall 2 rise 1
    server pg_secondary pg_secondary:5432 check inter 500ms fall 2 rise 1 backup
```

`haproxy_check` **user**: HAProxy sends a PostgreSQL startup message with this username. PostgreSQL responds with an authentication challenge (even "role does not exist" is a valid protocol-level response). The user needs no password, no schema access, and no database privileges. It is created by `haproxy-check-init.sql` via `withInitScript()` on each Testcontainers node; in the local docker-compose stack it is mounted as `/docker-entrypoint-initdb.d/init-haproxy-check.sql`.

---

6. Local Developer Environment (HAProxy + PgBouncer)

For manual failover testing without running a JVM test, a docker-compose stack is provided:

Files:

File	Purpose
`scripts/docker-compose-failover-local.yml`	Full stack definition
`scripts/haproxy-failover-local.cfg`	HAProxy config with stats page
`scripts/init-haproxy-check.sql`	Creates `haproxy_check` PostgreSQL user on first start

Topology:

```
Application (PeeGeeQ) or psql client
  │  host=localhost, port=6432
  ▼
PgBouncer:6432   (bitnami/pgbouncer:1.22.0)
```

```
    │  session pool, up to 200 client connections, 20 server connections
    │  connects to haproxy:5400 (not directly to PostgreSQL)
    ▼
HAProxy:5400     (haproxy:2.8-alpine, stats on :8404)
  ├──► pg-primary:5432    (postgres:15.13-alpine3.20, exposed :5433)
  └──► pg-secondary:5432  (postgres:15.13-alpine3.20, exposed :5434)
```

PgBouncer sits in front of HAProxy to handle the connection reset burst that occurs when HAProxy fails over: clients waiting for a connection receive a server-side disconnect; PgBouncer absorbs the reset and retries the server connection transparently to the client.

Stats page: `http://localhost:8404/stats` — shows live health of both backends.

Start:

```
docker compose -f scripts/docker-compose-failover-local.yml up -d
```

Simulate failover:

```
# Stop primary — HAProxy detects failure in ~1 s, routes to secondary
docker compose -f scripts/docker-compose-failover-local.yml stop pg-primary

# Connect through PgBouncer (or HAProxy direct on :5400)
psql -h localhost -p 6432 -U peegeeq_dev -d peegeeq_dev -c "SELECT 1"

# Restore primary — HAProxy auto-recovers, traffic returns to primary
docker compose -f scripts/docker-compose-failover-local.yml start pg-primary
```

---

7. Known Gaps

Five gaps exist between the current implementation and a fully resilient production setup. For detailed analysis, recommendations, and a phased implementation plan see PEEGEEQ_PG_CONNECTION_MANAGEMENT_HAPROXY_GAPS.md.

#	Gap	Risk
7.1	No circuit breaker on pool operations — failover window errors surface as raw TCP failures	MEDIUM
7.2	LISTEN/NOTIFY long-lived connection bypasses HAProxy — does not auto-recover after primary failover	LOW
7.3	Streaming replication not covered by the integration test	LOW
7.4	PgBouncer transaction pool mode with `search_path` tenant isolation is untested	LOW

#	Gap	Risk
7.5	Split-brain risk when streaming replication + automatic node promotion are combined without Patroni	LOW–MEDIUM

---

8. Using the pg-sidecar Service (`peegeeq-pg-sidecar`)

`peegeeq-pg-sidecar` is the project's built-in HTTP health-check sidecar. It exposes a single endpoint:

```
GET /primary
  → HTTP 200   when pg_is_in_recovery() = false  (this node is the write primary)
  → HTTP 503   when pg_is_in_recovery() = true   (replica) or PostgreSQL is
unreachable
```

Deploy one sidecar process alongside **each** PostgreSQL node. HAProxy calls the endpoint on the local sidecar to decide whether to route writes to that node.

8.1 Where it fits in the stack

```
PeeGeeQ application
  │  host=haproxy, port=5400    ← peegeeq.database.host / peegeeq.database.port
  ▼
HAProxy:5400
  ├──► pg-node-1:5432   check port 8008  httpchk GET /primary
  │       sidecar:8008 ──► pg-node-1:5432  SELECT pg_is_in_recovery()
  └──► pg-node-2:5432   check port 8008  httpchk GET /primary backup
          sidecar:8008 ──► pg-node-2:5432  SELECT pg_is_in_recovery()
```

The sidecar calls `pg_is_in_recovery()` on the local PostgreSQL node and reports the result to HAProxy. HAProxy only routes writes to the node whose sidecar returns HTTP 200.

8.2 Configuration

All settings are passed as JVM system properties (`-D`). No configuration file is required.

Property	Default	Description
`pg.host`	`localhost`	PostgreSQL host (should be `localhost` or the local node address)
`pg.port`	`5432`	PostgreSQL port
`pg.database`	`postgres`	Database to connect to
`pg.user`	`haproxy_check`	PostgreSQL user
`pg.password`	*(empty)*	Password (leave empty if using the no-password user setup)

Property	Default	Description
`http.port`	`8008`	Port the sidecar HTTP server listens on

The sidecar uses a pool of **2 connections** to the local node. No additional configuration is needed.

8.3 PostgreSQL user setup (one-time, per node)

The `haproxy_check` user requires only the ability to connect and call `pg_is_in_recovery()`. No schema access and no password are needed.

```
CREATE USER haproxy_check WITH PASSWORD '' CONNECTION LIMIT 3;
-- pg_is_in_recovery() is a built-in function; no GRANT is needed.
-- Optionally restrict to the postgres database only:
REVOKE CONNECT ON DATABASE postgres FROM PUBLIC;
GRANT  CONNECT ON DATABASE postgres TO haproxy_check;
```

In Testcontainers, supply this as an init script via `.withInitScript("haproxy-check-init.sql")`. In docker-compose, mount it as `/docker-entrypoint-initdb.d/init-haproxy-check.sql`.

8.4 Build and run

Fat-jar (JVM — any JDK 21+)

```
cd c:\Users\mraysmit\dev\idea-projects\peegeeq
mvn package -pl :peegeeq-pg-sidecar -DskipTests 2>&1 | Tee-Object -FilePath
logs\pg-sidecar-build.txt

java `
  -Dpg.host=localhost `
  -Dpg.port=5432 `
  -Dpg.database=postgres `
  -Dpg.user=haproxy_check `
  -Dpg.password= `
  -Dhttp.port=8008 `
  -jar peegeeq-pg-sidecar/target/peegeeq-pg-sidecar-1.0-SNAPSHOT.jar
```

Native binary (GraalVM JDK 21+)

```
mvn package -Pnative -pl :peegeeq-pg-sidecar -DskipTests 2>&1 | Tee-Object -
FilePath logs\pg-sidecar-native.txt
# Build takes 2–5 minutes.
# Output: peegeeq-pg-sidecar\target\peegeeq-pg-sidecar.exe  (Windows)
#         peegeeq-pg-sidecar/target/peegeeq-pg-sidecar       (Linux/macOS)
```

Run (Windows):

```
.\peegeeq-pg-sidecar\target\peegeeq-pg-sidecar.exe `
  -Dpg.host=localhost `
  -Dpg.port=5432 `
  -Dpg.user=haproxy_check `
  -Dpg.password= `
  -Dhttp.port=8008
```

Run (Linux/macOS):

```
./peegeeq-pg-sidecar/target/peegeeq-pg-sidecar \
  -Dpg.host=localhost \
  -Dpg.port=5432 \
  -Dpg.user=haproxy_check \
  -Dpg.password= \
  -Dhttp.port=8008
```

Container image (native binary, distroless, ~10 MB)

```
# Stage 1: build the native binary
FROM ghcr.io/graalvm/native-image-community:21 AS builder
WORKDIR /build
COPY . .
RUN mvn package -Pnative -pl :peegeeq-pg-sidecar -DskipTests

# Stage 2: distroless runtime — no JVM, no shell, minimal attack surface
FROM gcr.io/distroless/base-debian12
COPY --from=builder /build/peegeeq-pg-sidecar/target/peegeeq-pg-sidecar
/app/peegeeq-pg-sidecar
EXPOSE 8008
ENTRYPOINT ["/app/peegeeq-pg-sidecar"]
```

```
docker build -t peegeeq-pg-sidecar:latest .

docker run --rm \
  -e JAVA_TOOL_OPTIONS="-Dpg.host=db-node-1 -Dpg.port=5432 -Dpg.user=haproxy_check
-Dpg.password= -Dhttp.port=8008" \
  -p 8008:8008 \
  peegeeq-pg-sidecar:latest
```

8.5 HAProxy configuration for HTTP-based primary detection

Replace the `option pgsql-check` lines in the HAProxy config with `option httpchk` pointing at the sidecar. This gives HAProxy semantic knowledge of which node PostgreSQL considers the write primary, not just which node accepts TCP connections.

```
backend pg_write
    balance           leastconn
    option            httpchk GET /primary
    http-check        expect status 200

    # Deploy one sidecar alongside each PostgreSQL node, listening on port 8008.
    # HAProxy health-checks the sidecar HTTP endpoint; PostgreSQL port (5432) is
for data.
    server pg1 pg1:5432 check port 8008 inter 500ms fall 2 rise 1
    server pg2 pg2:5432 check port 8008 inter 500ms fall 2 rise 1 backup
```

HAProxy contacts `pg1:8008/primary` and `pg2:8008/primary` every 500 ms. Only the node whose sidecar returns HTTP 200 receives write traffic. The backup server becomes active only when the primary's sidecar returns HTTP 503 twice in a row (`fall=2`).

8.6 PeeGeeQ application configuration

Point PeeGeeQ at HAProxy, not at PostgreSQL directly. The sidecar and HAProxy together ensure only the write primary receives connections.

```
# peegeeq-default.properties (or environment-specific override)
peegeeq.database.host=haproxy-host     # HAProxy address, not the PostgreSQL node
peegeeq.database.port=5400             # HAProxy frontend port
peegeeq.database.name=peegeeq
peegeeq.database.username=peegeeq
peegeeq.database.password=peegeeq
peegeeq.database.schema=public
```

Or via system properties at startup:

```
java -Dpeegeeq.database.host=haproxy-host `
     -Dpeegeeq.database.port=5400 `
     -jar peegeeq-runtime/target/peegeeq-runtime.jar
```

The `PgConnectionManager` pool is built once from these values. All connection attempts, reconnects, and failovers happen transparently through the HAProxy address — no code change or pool restart is required when PostgreSQL failover occurs.

8.7 Verifying the sidecar

While the sidecar is running:

```
# PowerShell
Invoke-WebRequest -Uri http://localhost:8008/primary -Method GET | Format-List
StatusCode, StatusDescription
# Primary node  → StatusCode 200
# Replica node  → StatusCode 503

# curl
curl -o /dev/null -s -w "%{http_code}\n" http://localhost:8008/primary
```

Unknown paths return HTTP 404:

```
Invoke-WebRequest -Uri http://localhost:8008/health -Method GET
# → 404
```

8.8 Running the integration test

The integration test (`PgPrimaryCheckIntegrationTest`) starts a real PostgreSQL container, deploys the verticle, and verifies all three response cases (200, 404, 503):

```
cd c:\Users\mraysmit\dev\idea-projects\peegeeq
mvn test -pl :peegeeq-pg-sidecar -Pintegration-tests 2>&1 | Tee-Object -FilePath
logs\pg-sidecar-integration-20260510.txt
```

Requirements: Docker Desktop running. The test uses `postgres:15.13-alpine3.20` (pulled once, cached by Docker). Expected run time: under 30 seconds.

Appendix A: HAProxy Primary Detection Options

The following is the full content of peegeeq-service-manager/docs/PG_HAPROXY_PRIMARY_DETECTION_OPTIONS.md.

HAProxy PostgreSQL Routing Without Patroni

Author: Mark A Ray-Smith Cityline Ltd.
Date: May 10, 2026
Status: REFERENCE
Module: `peegeeq-service-manager`

---

1. The Core Problem

HAProxy is a battle-tested TCP/HTTP load balancer commonly placed in front of a PostgreSQL primary-replica pair to give the application a single, stable connection endpoint that survives a node failure.

`option pgsql-check` and plain TCP checks both verify that PostgreSQL is **alive**. Both the primary and any replica pass these checks. Neither tells HAProxy which node is the **write primary**.

Only something that queries `SELECT pg_is_in_recovery()` can distinguish them:

Node role	`pg_is_in_recovery()`	HAProxy should…
Primary (read-write)	`f`	Route writes here
Standby / replica	`t`	Block writes / mark backup

This is precisely what Patroni's `/primary` HTTP endpoint does internally — it calls `pg_is_in_recovery()` and returns HTTP 200 for primary, HTTP 503 for replica.

---

2. What Patroni Does

Understanding what Patroni provides is necessary context for evaluating the alternatives.

Patroni is a Python daemon that runs on each PostgreSQL node and does three distinct things:

2.1 Leader Election via a Distributed Data Store (DCS)

Patroni stores the identity of the current primary in an external DCS — etcd, Consul, or ZooKeeper. Each Patroni instance holds a **session-based lock** (a TTL key in etcd, or a Consul session + KV key). The primary continuously renews that lock. When the primary fails:

1. The lock TTL expires.
2. The standby with the most up-to-date WAL position races to acquire the lock.
3. The winner of the race is authorised to promote.

This is the property the alternatives in §3 either replicate (Consul-based monitor), partially replicate (repmgr with witness), or omit entirely (custom HTTP sidecar).

2.2 Automatic Promotion

Once a Patroni standby wins the DCS lock it calls:

```
pg_promote()          ← SQL, PostgreSQL 12+
  or
pg_ctl promote        ← shell command, older PostgreSQL
```

It then updates the DCS entry to announce itself as the new primary.

2.3 HTTP Status Endpoint for HAProxy

Patroni runs a small REST API on each node (default port 8008).

Endpoint	HTTP 200	HTTP 503
`/primary`	This node is the write primary	This node is not the primary
`/replica`	This node is a healthy replica	This node is not a replica
`/health`	Node is running	Node is unhealthy

HAProxy uses `option httpchk GET /primary` to query this endpoint. Servers that return 503 are taken out of rotation. This is the only part of Patroni that HAProxy interacts with.

2.4 Fencing the Old Primary

When the old primary comes back after a crash it must be prevented from accepting writes until it re-syncs. Patroni handles this by:

- Refusing to start PostgreSQL on the old primary until it has confirmed with the DCS that another node now holds the leader lock.
- Running `pg_rewind` automatically to align the old primary's WAL with the new primary's timeline before starting replication.

This is the hardest part to replicate without Patroni. Without it, a recovered old primary can accept writes concurrently with the new primary, causing data divergence.

2.5 Summary of What Patroni Provides

Capability	Patroni provides	Notes
HAProxy routing signal (`/primary` HTTP)	Yes	Trivially replicable with a sidecar
Automatic promotion	Yes	Requires DCS or equivalent
DCS-backed leader election (no split-brain)	Yes	Core value of Patroni
Automatic fencing / pg_rewind on re-join	Yes	Hardest to replicate
Monitoring REST API (`/patroni`, `/history`)	Yes	Nice-to-have, not critical for routing

---

3. Options Without Patroni

3.1 Custom HTTP Sidecar (Patroni-Equivalent, Minimal)

Patroni's `/primary` endpoint is not complex. The logic is:

```
GET /primary
  → SELECT pg_is_in_recovery()
  → false  → HTTP 200  (this node is the write primary)
  → true   → HTTP 503  (this node is a replica)
```

You can replicate this with a small process on each PostgreSQL node. Since PeeGeeQ is a Vert.x project, a Java implementation is the natural fit: no extra language runtime, packaged as a standard fat-jar or native image,

consistent with the rest of the codebase.

Java / Vert.x implementation (runs on each node, listens on port 8008):

```
// PgPrimaryCheckVerticle.java — deploy as a standalone Vert.x verticle
// Exposes GET /primary → 200 if write primary, 503 if replica or unreachable
public class PgPrimaryCheckVerticle extends AbstractVerticle {

    private static final int HTTP_PORT = 8008;

    @Override
    public void start(Promise<Void> start) {
        PgConnectOptions connectOptions = new PgConnectOptions()
                .setHost(config().getString("pg.host", "localhost"))
                .setPort(config().getInteger("pg.port", 5432))
                .setDatabase(config().getString("pg.database", "postgres"))
                .setUser(config().getString("pg.user", "haproxy_check"))
                .setPassword(config().getString("pg.password", ""));

        Pool pool = PgBuilder.pool()
                .with(new PoolOptions().setMaxSize(2))
                .connectingTo(connectOptions)
                .using(vertx)
                .build();

        vertx.createHttpServer()
                .requestHandler(req -> {
                    if ("/primary".equals(req.path())) {
                        pool.query("SELECT pg_is_in_recovery()")
                                .execute()
                                .onSuccess(rows -> {
                                    boolean isReplica =
rows.iterator().next().getBoolean(0);
                                    req.response().setStatusCode(isReplica ? 503 :
200).end();
                                })
                                .onFailure(err ->
req.response().setStatusCode(503).end());
                    } else {
                        req.response().setStatusCode(404).end();
                    }
                })
                .listen(HTTP_PORT)
                .<Void>mapEmpty()
                .onComplete(start);
    }
}
```

Main launcher (for running as a standalone process):

```
public class PgPrimaryCheckMain {
    public static void main(String[] args) {
        Vertx vertx = Vertx.vertx();
        JsonObject config = new JsonObject()
                .put("pg.host",     System.getProperty("pg.host", "localhost"))
                .put("pg.port",     Integer.getInteger("pg.port", 5432))
                .put("pg.database", System.getProperty("pg.database", "postgres"))
                .put("pg.user",     System.getProperty("pg.user",
"haproxy_check"))
                .put("pg.password", System.getProperty("pg.password", ""));

        vertx.deployVerticle(new PgPrimaryCheckVerticle(),
                new DeploymentOptions().setConfig(config));
    }
}
```

Run:

```
java -Dpg.host=localhost -Dpg.port=5432 -Dpg.user=haproxy_check \
     -jar pg-primary-check.jar
```

Why Java over Python here:

- No additional language runtime on the node — the JVM is already required for PeeGeeQ.
- Uses `io.vertx.pgclient` (same driver as PeeGeeQ) — consistent behaviour, no new PostgreSQL dependency.
- **Built as a GraalVM native image** for production deployments: instant startup (~10 ms), minimal memory footprint (~15 MB RSS), no JVM on the node required. The fat-jar form is retained for development and debugging. See §7.3 for the native build command.
- The `haproxy_check` user needs no password and only the `SELECT pg_is_in_recovery()` privilege — same as the Python version.

HAProxy config (identical regardless of sidecar language):

```
backend pg_write
    option httpchk GET /primary
    server pg1 pg1:5432 check port 8008 inter 500ms fall 2 rise 1
    server pg2 pg2:5432 check port 8008 backup inter 500ms fall 2 rise 1
```

This is functionally identical to what Patroni exposes. The HTTP check and HAProxy configuration are the same; the only thing absent is the DCS-backed leader election that Patroni adds on top.

Deployment note: this verticle must run as a sidecar on each PostgreSQL host (or container). In a container environment it can run in a sidecar container sharing the network namespace with PostgreSQL, or as a second process in the same container managed by a simple process supervisor (e.g. `s6`, `supervisord`).

3.2 HAProxy `agent-check` (TCP, No HTTP Needed)

HAProxy has a dedicated `agent-check` mechanism. An agent process listens on a TCP port; HAProxy connects, sends nothing, and reads a single text response:

Agent response	HAProxy action
`up\n`	Mark server UP, route traffic
`down\n`	Mark server DOWN, stop routing
`drain\n`	Drain existing connections, stop new ones

Minimal shell agent (served via `xinetd` or `socat` on port 9000 of each node):

```
#!/bin/bash
# pg_agent.sh
result=$(psql -U haproxy_check -Atc "SELECT pg_is_in_recovery()")
if [ "$result" = "f" ]; then
    echo "up"
else
    echo "down"
fi
```

`socat` **wrapper** to expose the script on a TCP port:

```
socat TCP-LISTEN:9000,fork,reuseaddr EXEC:/opt/pg_agent.sh
```

HAProxy config:

```
backend pg_write
    server pg1 pg1:5432 check agent-check agent-port 9000 agent-inter 500ms
    server pg2 pg2:5432 check agent-check agent-port 9000 agent-inter 500ms backup
```

The `agent-check` and TCP health check run independently. Both must pass for HAProxy to consider a server available.

3.3 `repmgr` + `repmgrd` (Automated Promotion, No DCS)

`repmgr` is the traditional PostgreSQL replication manager. The `repmgrd` daemon handles automatic failover without requiring an external distributed data store (etcd, Consul, ZooKeeper).

Key characteristics:

- `repmgrd` runs on each PostgreSQL node.
- It monitors the primary and triggers promotion scripts when the primary is unresponsive.

- A **witness server** (a lightweight third node, not a full PostgreSQL replica) is recommended to provide quorum and prevent split-brain; without it a two-node cluster has the same split-brain exposure as any other two-node setup.
- `repmgr` does not expose an HTTP endpoint compatible with `option httpchk`; HAProxy integration still requires one of the sidecar approaches above.
- Automatic failover is optional — `repmgr` can be used for monitoring and manual failover only.

---

4. Comparison

Approach	Complexity	Extra process required	Automatic promotion	Split-brain safe
Custom HTTP sidecar (§3.1)	Very low	Yes — `peegeeq-pg-sidecar` (Vert.x Java fat-jar)	No	No
HAProxy `agent-check` (§3.2)	Low	Yes — shell + socat/xinetd	No	No
repmgr + repmgrd (§3.3)	Medium	Yes — repmgrd daemon	Yes (with witness)	Partial
Consul-based monitor (see `PG_FAILOVER_CONSUL_DESIGN.md`)	Medium	No (uses existing Consul)	Yes	Yes (with fencing)
Patroni	High	Yes — Patroni + DCS	Yes	Yes

---

5. What Each Option Provides and Lacks

Custom HTTP sidecar / `agent-check`

Provides:

- HAProxy routes writes only to the current primary — correctly.
- Identical behaviour to Patroni from HAProxy's perspective.
- No new runtime dependencies.

Does not provide:

- Automatic promotion of the standby when the primary fails.
- Split-brain prevention — if you promote manually and the old primary recovers, both nodes could accept writes until the operator intervenes.

Appropriate when:

- Promotion is manual (operator decides when to promote).
- Downtime during primary failure is acceptable until operator action.
- You want the simplest possible routing-correctness fix.

`repmgr` + `repmgrd`

Provides:

- Automatic promotion without Patroni.
- History of switchovers/failovers stored in `repmgr` schema.

Does not provide:

- Consul/etcd-backed quorum (uses its own TCP-based primary check).
- Guaranteed split-brain safety without a witness node.

Appropriate when:

- You want automatic promotion without adding Consul or etcd.
- You can provision a third node (witness) for quorum.

Consul-based monitor (PeeGeeQ-native)

See `PG_FAILOVER_CONSUL_DESIGN.md` for the full design. This is the preferred path for PeeGeeQ because Consul is already present in `peegeeq-service-manager` and the integration is fully reactive (Vert.x `io.vertx.ext.consul.ConsulClient`).

---

6. Recommendation for PeeGeeQ

Scenario	Recommended approach
Development / single-node	`option pgsql-check` already in HAProxy test config — sufficient
Production, manual failover acceptable	Custom HTTP sidecar (§2.1) + HAProxy `httpchk`
Production, automatic failover required	Consul-based monitor (`PG_FAILOVER_CONSUL_DESIGN.md`)
Production, no Consul, 3 nodes available	`repmgr` + witness node

---

7. Building the Java Sidecar

The sidecar is implemented in `peegeeq-pg-sidecar`. Two deployment artefacts are supported: a **fat-jar** (JVM) and a **native binary** (GraalVM). Both expose the same `/primary` endpoint and accept the same system property configuration.

7.1 Prerequisites

Artefact	Requirement
Fat-jar	JDK 21+ (any distribution)
Native binary	GraalVM JDK 21+ with `native-image` installed

Install GraalVM and `native-image` (one-time):

```
# Option A — SDKMAN (Linux / macOS / WSL)
sdk install java 21.0.3-graalce
gu install native-image

# Option B — Winget (Windows, native GraalVM distribution)
winget install GraalVM.GraalVM.Community.21

# Option C — download directly from https://www.graalvm.org/downloads/
# Then add GRAALVM_HOME/bin to PATH and run:
gu install native-image
```

Verify:

```
java -version              # should show GraalVM
native-image --version     # should print GraalVM native-image version
```

7.2 Build the Fat-Jar (JVM)

```
cd c:\Users\mraysmit\dev\idea-projects\peegeeq
mvn package -pl :peegeeq-pg-sidecar -DskipTests 2>&1 | Tee-Object -FilePath
logs\pg-sidecar-build.txt
```

Output: `peegeeq-pg-sidecar/target/peegeeq-pg-sidecar-1.0-SNAPSHOT.jar`

Run:

```
java `
  -Dpg.host=localhost `
  -Dpg.port=5432 `
  -Dpg.database=postgres `
  -Dpg.user=haproxy_check `
  -Dpg.password= `
  -Dhttp.port=8008 `
  -jar peegeeq-pg-sidecar/target/peegeeq-pg-sidecar-1.0-SNAPSHOT.jar
```

7.3 Build the Native Binary (GraalVM)

```
cd c:\Users\mraysmit\dev\idea-projects\peegeeq
mvn package -Pnative -pl :peegeeq-pg-sidecar -DskipTests 2>&1 | Tee-Object -
FilePath logs\pg-sidecar-native.txt
```

The build takes 2–5 minutes. Output:

- Windows: `peegeeq-pg-sidecar/target/peegeeq-pg-sidecar.exe`
- Linux/macOS: `peegeeq-pg-sidecar/target/peegeeq-pg-sidecar`

Run (Windows):

```
.\peegeeq-pg-sidecar\target\peegeeq-pg-sidecar.exe `
  -Dpg.host=localhost `
  -Dpg.port=5432 `
  -Dpg.user=haproxy_check `
  -Dpg.password= `
  -Dhttp.port=8008
```

Run (Linux/macOS):

```
./peegeeq-pg-sidecar/target/peegeeq-pg-sidecar \
  -Dpg.host=localhost \
  -Dpg.port=5432 \
  -Dpg.user=haproxy_check \
  -Dpg.password= \
  -Dhttp.port=8008
```

7.4 Container Image (Native Binary, Distroless)

Build a minimal container image from the native binary using a two-stage Dockerfile. The final image contains only the binary — no JVM, no shell, no package manager.

```
# Stage 1: build the native binary
FROM ghcr.io/graalvm/native-image-community:21 AS builder
WORKDIR /build
COPY . .
RUN mvn package -Pnative -pl :peegeeq-pg-sidecar -DskipTests

# Stage 2: distroless runtime — minimal attack surface, ~10 MB image
FROM gcr.io/distroless/base-debian12
COPY --from=builder /build/peegeeq-pg-sidecar/target/peegeeq-pg-sidecar
/app/peegeeq-pg-sidecar
EXPOSE 8008
ENTRYPOINT ["/app/peegeeq-pg-sidecar"]
```

Build and run:

```
docker build -t peegeeq-pg-sidecar:latest .

docker run --rm \
```

```
    -e pg.host=db-node-1 \
    -e pg.port=5432 \
    -e pg.user=haproxy_check \
    -e pg.password= \
    -p 8008:8008 \
    peegeeq-pg-sidecar:latest
```

Note: system properties (`-D`) do not pass through environment variables automatically in a native binary. If running in a container, you can either set the properties in `ENTRYPOINT` or add a small config-from-env layer to `PgPrimaryCheckMain`.

7.5 PostgreSQL User Setup

The `haproxy_check` user requires no password and only the minimum privilege needed to call `pg_is_in_recovery()`:

```
CREATE USER haproxy_check WITH PASSWORD '' CONNECTION LIMIT 3;
-- pg_is_in_recovery() is a built-in function; no GRANT is needed.
-- Optionally restrict to the postgres database only:
REVOKE CONNECT ON DATABASE postgres FROM PUBLIC;
GRANT  CONNECT ON DATABASE postgres TO haproxy_check;
```

7.6 Verify the Endpoint

```
# While the sidecar is running:
Invoke-WebRequest -Uri http://localhost:8008/primary -Method GET | Select-Object
StatusCode
# Primary node  → StatusCode 200
# Replica node  → StatusCode 503
```

Or with `curl`:

```
curl -o /dev/null -s -w "%{http_code}\n" http://localhost:8008/primary
```

End of document.

Appendix B: The JDBC Multi-Host Failover Pattern (Historical Reference)

This appendix documents a client-side failover pattern that exists in the JDBC world. **It is not applicable to PeeGeeQ** and is recorded here only as context for readers who have encountered it in older systems.

The Pattern

The PostgreSQL JDBC driver (version 42.2+) supports a multi-host connection URL:

```
jdbc:postgresql://host1:5432,host2:5432/database?targetServerType=primary
```

The driver iterates the host list on each connection attempt, queries `pg_is_in_recovery()` internally, and connects to the first node matching `targetServerType`:

`targetServerType`	**Behaviour**
`primary`	Only the write primary
`preferPrimary`	Primary first, any node as fallback
`secondary`	Only a replica
`any`	First available node

No proxy process, no sidecar, no DCS — the failover logic lives entirely inside the JDBC driver.

Why It Is Not Used in PeeGeeQ

Not applicable to PeeGeeQ: PeeGeeQ is designed as a pure reactive system built on Vert.x. JDBC is a blocking, synchronous protocol — every query occupies a thread until a response arrives. In a reactive event-loop architecture, blocking a thread stalls the entire loop and defeats the purpose of the non-blocking model. PeeGeeQ therefore forbids JDBC entirely: `DriverManager`, `PreparedStatement`, `ResultSet`, and JDBC URL strings are all banned. PeeGeeQ uses the Vert.x reactive pgclient (`io.vertx.pgclient`) exclusively, which configures hosts via `PgConnectOptions`, not a URL string. The reactive client has no built-in equivalent of `targetServerType`.

Why It Is Not a Sound Pattern for Distributed Systems

Even in JDBC codebases, this pattern has structural problems that make it unsuitable for modern microservice architectures:

1. **Uncoordinated failover across services**: each microservice instance independently iterates the host list on its next connection attempt. After a primary failure, different service instances may connect to different nodes during the promotion window — some still seeing the old primary (if it partially recovers), others reaching the new primary. The application cluster has no shared view of which node is authoritative.

2. **No fencing**: the driver cannot fence the old primary. If the old primary comes back before promotion is complete, some connections will re-attach to it and accept writes concurrently with the new primary — data divergence.

3. **Per-process host-list management**: every service process must be configured with the full host list. Adding or removing a PostgreSQL node requires a configuration change and restart of every service instance. In contrast, a single HAProxy instance absorbs the topology change without touching application configuration.

4. **Promotion race**: the driver retries connection to the next host only when the current attempt fails. During the promotion window (old primary gone, new primary not yet accepting connections) all instances will exhaust their retry loop simultaneously, creating a thundering-herd reconnect storm against the new primary.

5. **No semantic health signal for routing**: the driver's host iteration is connection-scoped. Once connected, there is no ongoing check that the chosen node remains the primary. A long-lived connection pool will silently keep connections to a node that has been demoted, routing write queries to a replica that will reject them with `ERROR: cannot execute ... in a read-only transaction`.

Conclusion: an external proxy (HAProxy) or a monitor with a distributed lock (Consul) is the correct abstraction boundary for database-tier failover in a multi-service architecture. The proxy is the single, shared point of truth about which node is the primary; all service instances benefit automatically without any per-service failover logic.